

Managing Overlay Networks

➤ Virtual edge

- TCP connection
- or simply an pair of IP addresses

➤ Overlay maintenance

- Periodically ping to make sure neighbor is still alive
- Or verify aliveness while messaging
- If neighbor goes down, may want to establish new edge
- New incoming node needs to bootstrap
- **Could be a challenge under high rate of churn**

Key Research Problems

- Lookup
 - How to find out the appropriate content/resource that a user wants
- Management
 - How to maintain the P2P system under high rate of churn efficiently
 - Application reliability is difficult to guarantee
- Throughput
 - Content distribution/dissemination applications
 - How to copy content fast, efficiently, reliably

More About Lookup

- Centralized vs. decentralized
- How do you locate data/files/objects in a large P2P system built around a dynamic set of nodes in a scalable manner **without** any centralized server or hierarchy?
- Efficient routing even if the structure of the network is unpredictable
 - **Some P2P networks impose no meaningful structure**

Agenda

- Overview on P2P Networks
- P2P Applications
- Properties of P2P Systems
- Structured and Unstructured P2P
- Distributed Hash Table (DHT)

Unstructured and Structure P2P Systems

- **Unstructured P2P networks** allow resources to be placed at any node. The network topology is arbitrary, and the growth is spontaneous ← a.k.a. Mesh-based
- **Structured P2P networks** simplify resource location and load balancing by defining a topology and defining rules for resource placement ← a.k.a. Tree-based
- Need ways to **efficiently search for objects**

Q: What are the possible solutions? Gossip protocols or DHT?

Sample Unstructured P2P System: BitTorrent

➤ Pros

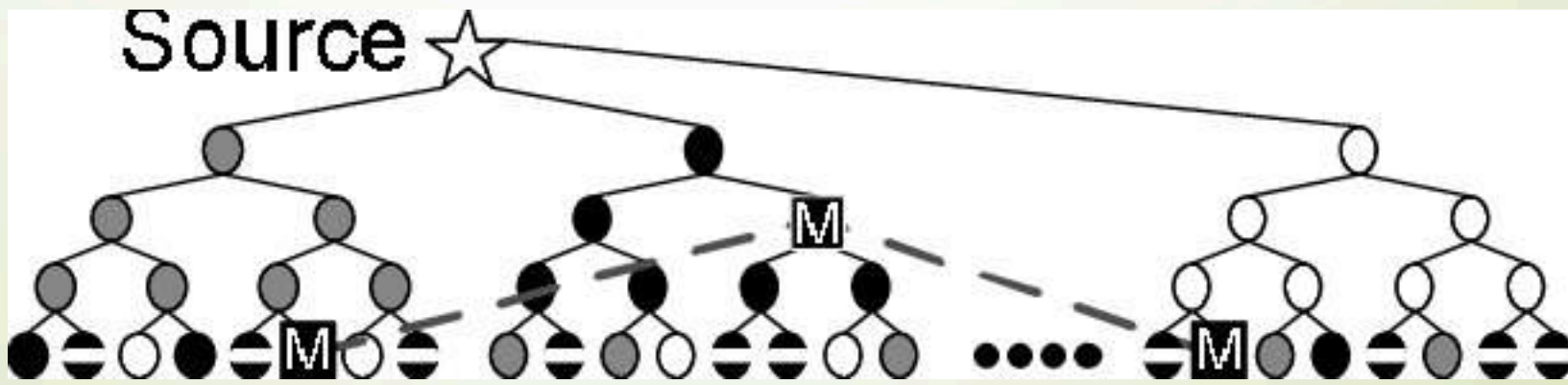
- Proficient in utilizing partially downloaded files
- Encourages diversity through “rarest-first”
 - Extends lifetime of swarm
- Works well for “hot content”

➤ Cons

- Assumes all interested peers active at same time; performance deteriorates if swarm “cools off”
- Even worse: no trackers for obscure content

Sample Structured P2P System: SplitStream

- Forest based dissemination
- Basic idea
 - Split the stream into K stripes (with MDC coding)
 - For each stripe create a multicast tree such that the forest
 - Contains interior-node-disjoint trees
 - Respects nodes' individual bandwidth constraints

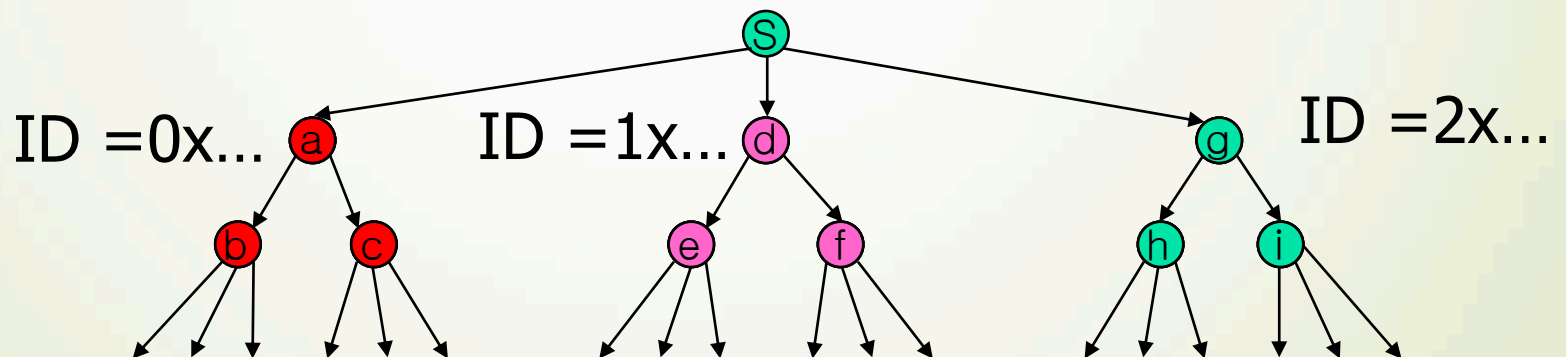


Multiple Description Coding (MDC)

- Multiple Description coding
 - Fragments a single media stream into M substreams ($M \geq 2$)
 - K packets are enough for decoding ($K < M$)
 - Less than K packets can be used to **approximate** content
 - Useful for multimedia (video, audio)
 - but not for other data, which needs erasure coding

Interior-Node-Disjoint Tree

- Each node in a set of trees is interior node in at most one tree and leaf node in the other trees.
- Each substream is disseminated over subtrees
- **Q: What's the intuition behind this?**



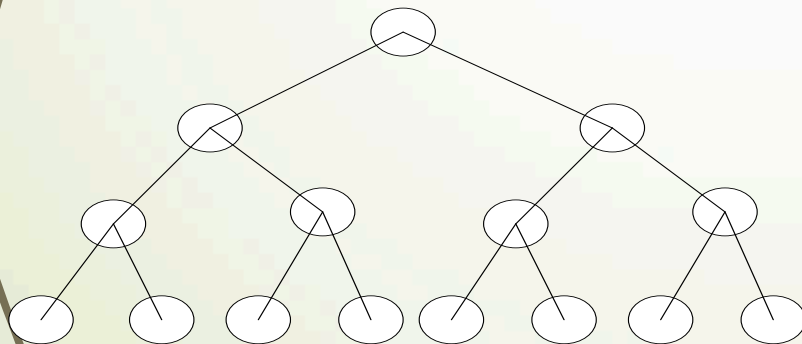
Constructing Forests

- Each stream has its **groupID**
 - Each groupID starts with a different digit
- A subtree is formed by the routes from all members to the groupID
 - The nodeIDs of all interior nodes share some number of starting digits with the subtree's groupID
- All nodes have **incoming capacity requirements** (number of stripes they need) and **outgoing capacity limits**

Pros/Cons of Unstructured and Structured P2P

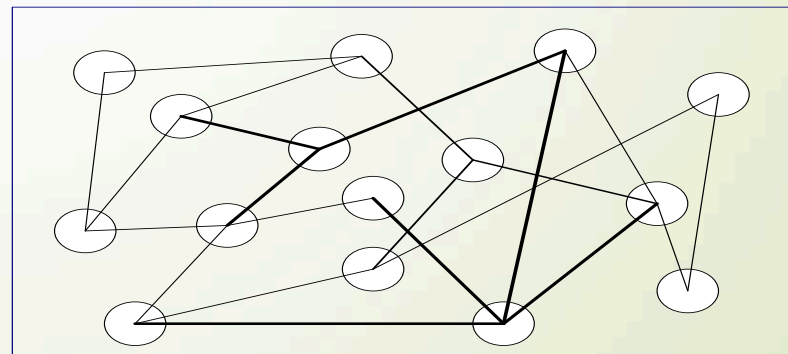
Tree:

- Intuitive way to implement a decentralized solution
- Logic is built into the structure of the overlay
- Sophisticated mechanisms for heterogeneous networks
- Fault-tolerance Issues



Mesh-Based:

- Multiple overlay links
- High-BW peers: more connections
- Robust to failures
 - Find new neighbors when links are broken
 - Chunks are sent via multiple paths
- Simpler to implement
- **Q: What is the downside?**



Unique Challenges of Structured P2P

- ➡ ID (node, key) mapping
- ➡ Routing (Lookup) method
- ➡ Maintenance (Join/Leave) method
- ➡ All functionality should be fully distributed

Summary : Unstructured vs Structured

	Query Lookup	Overlay Network Management
Unstructured	Gossip-based (heavy overhead)	Simple
Structured	Bounded and efficient ← DHT (discussed next)	Complex (heavy overhead)

Agenda

- Overview on P2P Networks
- P2P Applications
- Properties of P2P Systems
- Structured and Unstructured P2P
- Distributed Hash Table (DHT)

Distributed Hash Table (DHT)

- DHT: distributed P2P database
- database has (key, value) pairs;
 - key: ss number; value: human name
 - key: content type; value: IP address
- peers query DB with key
 - DB returns values that match the key
- peers can also insert (key, value)
peers

DHT Identifiers (Sample Solution)

- assign integer identifier to each peer in range $[0, 2^n - 1]$.
 - Each identifier can be represented by n bits.
- require each key to be an integer in **same range**.
- to get integer keys, hash original key.
 - e.g., key = $h(\text{"Led Zeppelin IV"})$
 - this is why they call it a distributed "hash" table

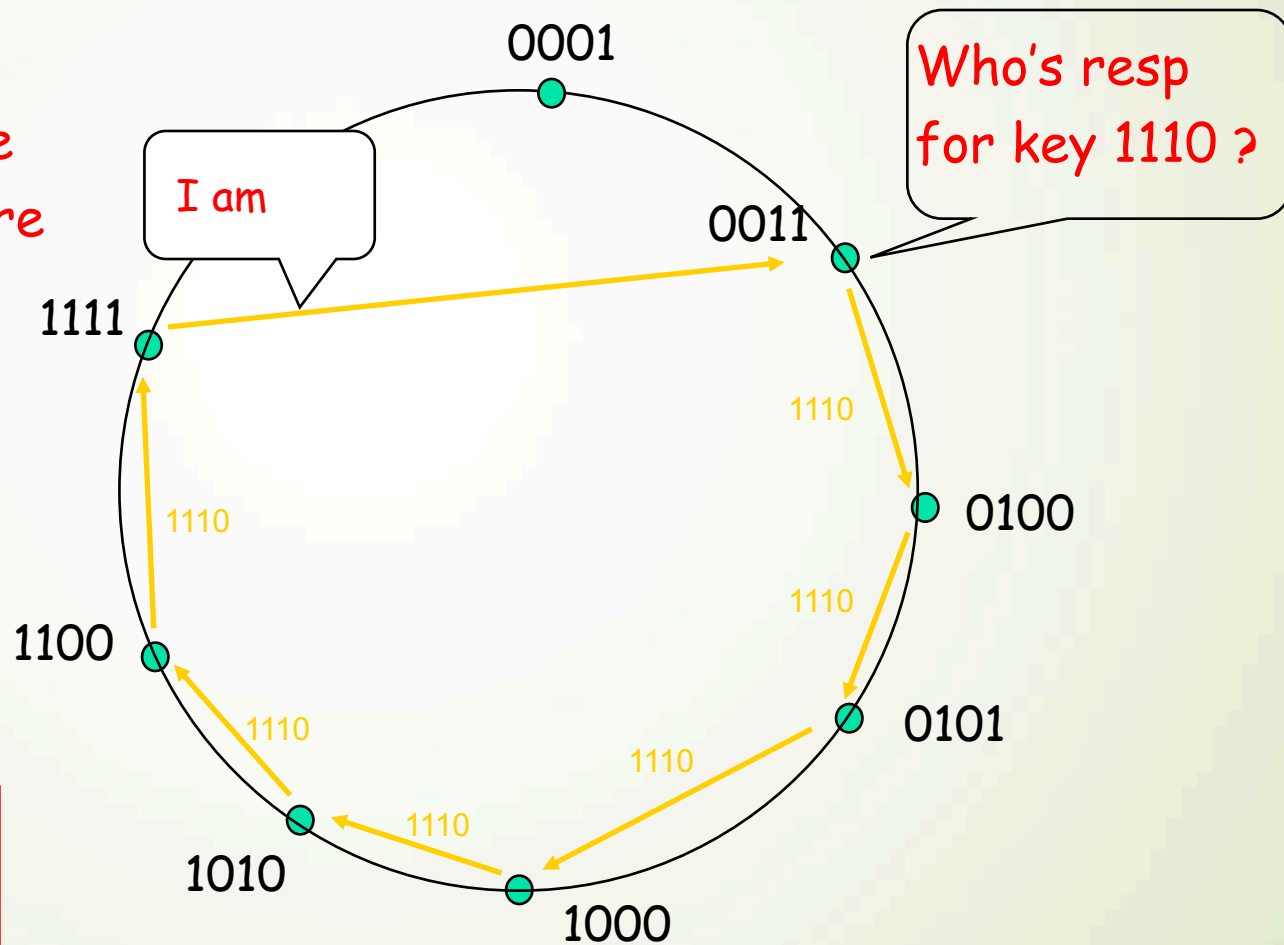
How to Assign Keys to Peers?

- core issue:
 - assigning (key, value) pairs to peers.
- rule: assign key to the peer that has the **closest** ID.
- convention in the literature: closest is the **immediate successor** of the key.
- e.g.,: $n=4$; peers: 1,3,4,5,8,10,12,14;
 - key = 13, then successor peer = 14
 - key = 15, then successor peer = 1

Circular DHT

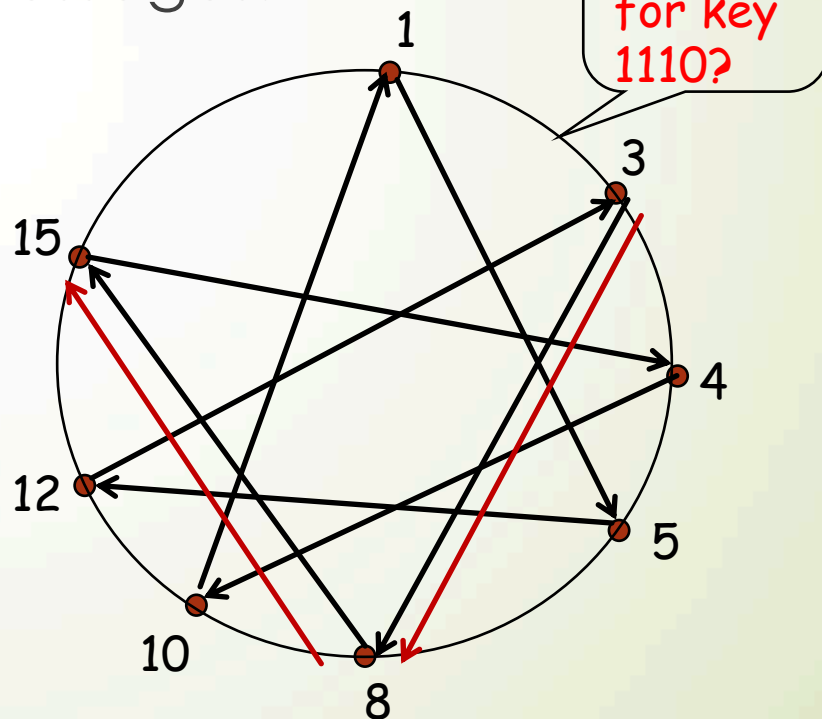
$O(N)$ messages
on avg to resolve
query, when there
are N peers

Define closest
as closest
successor



Circular DHT with Shortcuts

- Idea: keep more pointers (for shortcuts)
- each peer keeps track of IP addresses of predecessor, successor, short cuts.
- reduced from 6 to 2 messages.
- possible to design shortcuts so $O(\log N)$ neighbors, $O(\log N)$ messages in query



More Structured P2P Systems

- Chord

- Consistent hashing based ring structure

- Pastry

- Uses ID space concept similar to Chord
 - Exploits concept of a nested group

- CAN

- Nodes/objects are mapped into a d-dimensional Cartesian space

- Kademlia

- Similar structure to Pastry, but the method to check the closeness is XOR function

Questions

chsu@cs.nthu.edu.tw

